



BITS Pilani
Pilani Campus

Digital Design

Prof. GSS Chalapathi,
Prof. Saurabh Gandhi,
Prof. Ramesh Sambangi

Review questions

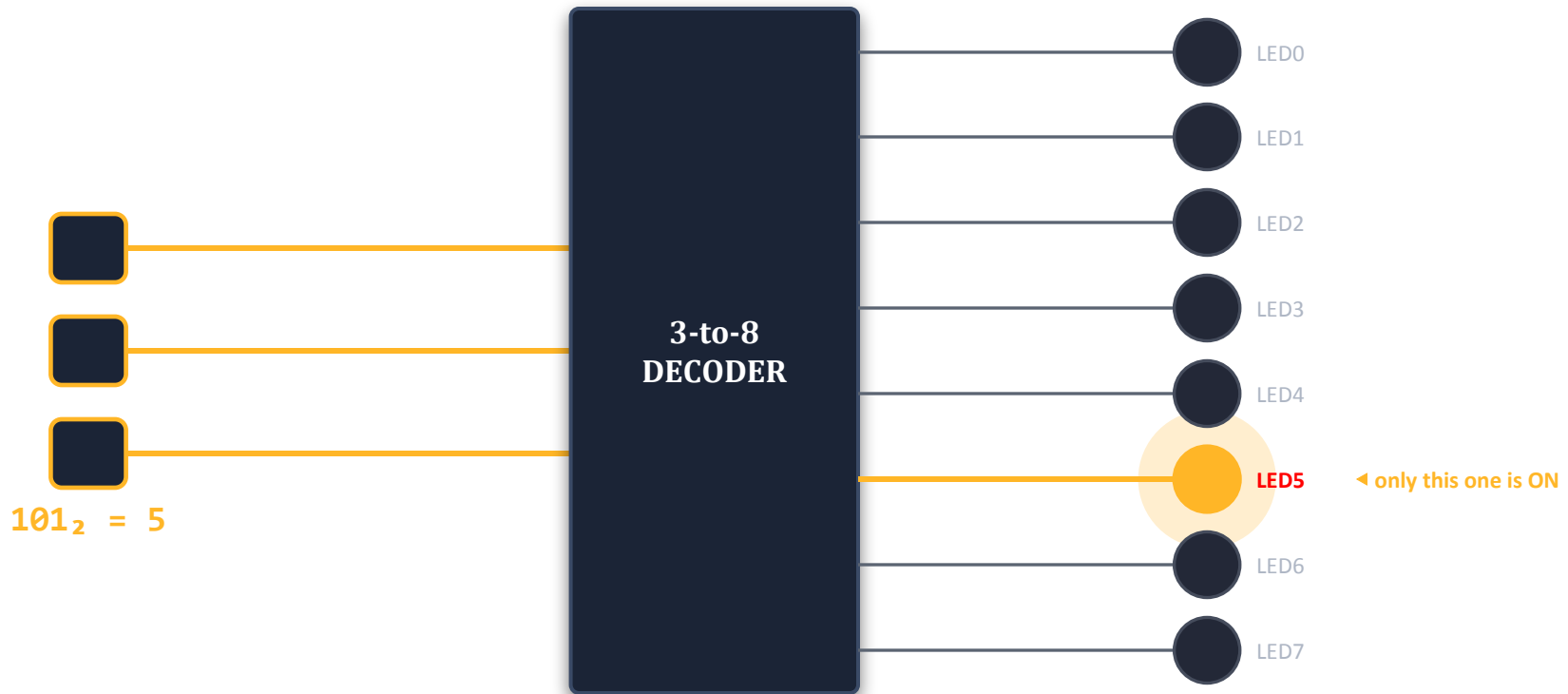
1. Final expression after simplification with k-maps must include what terms?
2. While applying k-maps, don't cares should be treated as?

A circuit to turn on the right LED

Turn on exactly **1 of 8** LEDs, depending on a certain stored value.

How many bits do we need to store the value of the LED to be turned on?

How many output wires does the circuit need to turn on the correct LED?

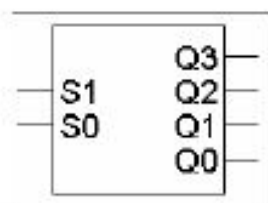


A decoder expands a compact **3-bit** code back into **8 individual output lines** — exactly one active at a time.

Decoders



- A **n -to- 2^n decoder** uses its n -bit input to determine which of 2^n outputs will be uniquely activated.



S1	S0	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

- Here is a block diagram and truth table for a **2-to-4 decoder**.
 - The two-bit input is called **S1S0**, and the four outputs are **Q0-Q3**.
 - If the input is the binary number i , then output Q_i alone will be true.
- This circuit “decodes” a binary number into a “one-of-four” code.

Building a decoder

- We can use the truth table to derive minimal sum of products equations for each of the four outputs (Q0-Q3), based on the two inputs (S0-S1).

S1	S0	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

- In this case there's not much to be simplified. Here are the equations:

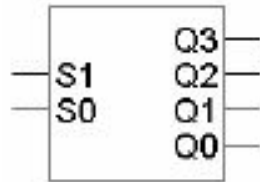
$$Q0 = S1'S0'$$

$$Q1 = S1'S0$$

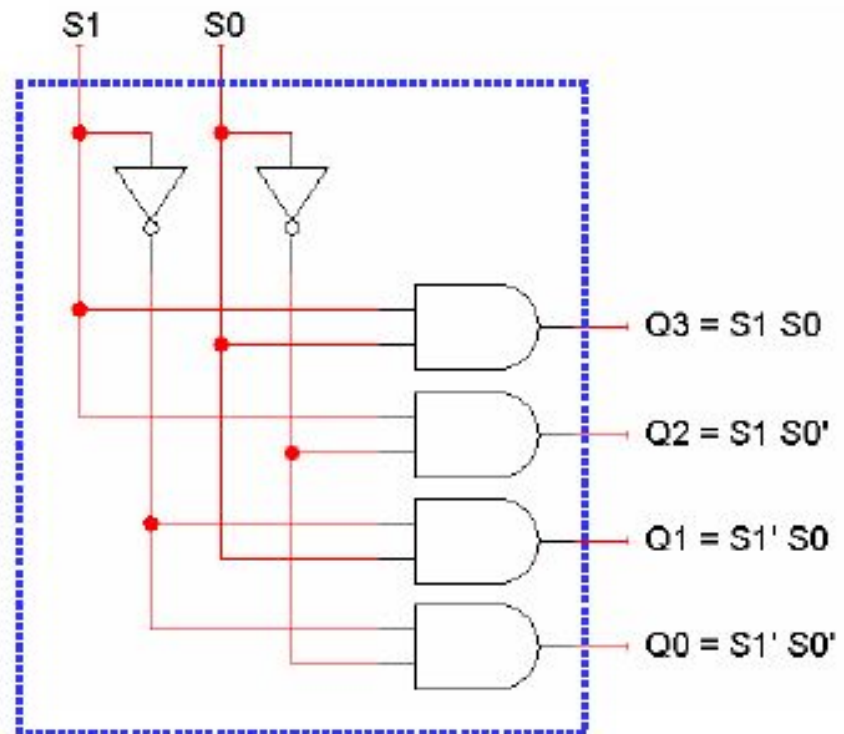
$$Q2 = S1 S0'$$

$$Q3 = S1 S0$$

Building a decoder – the circuit

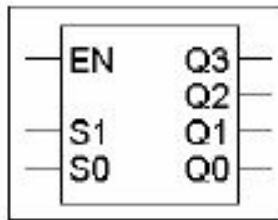


S1	S0	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



Decoders with 'enable' inputs

- Decoders can include an *enable* input (*EN*)
- $EN = 0$ disables the decoder (all outputs are 0)
- $EN = 1$ enables the decoder (exactly 1 output is 1 depending on the input)

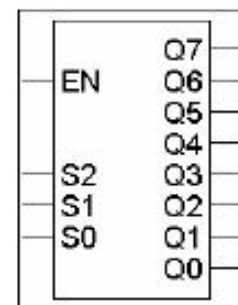


EN	S1	S0	Q0	Q1	Q2	Q3
0	x	x	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

3 – to – 8 decoder



- Larger decoders are similar. Here is a 3-to-8 decoder.
 - There are three selection inputs $S_2S_1S_0$, which activate one of eight outputs, Q_0 - Q_7 .
 - Again, only one output will be true for any input combination.
- A truth table and output equations for a 3-to-8 decoder (without EN) are given below.



S2	S1	S0	Q0	Q1	Q2	Q3	Q4	Q5	Q6	Q7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

$$Q_0 = S_2'S_1'S_0'$$

$$Q_1 = S_2'S_1S_0'$$

$$Q_2 = S_2'S_1S_0$$

$$Q_3 = S_2S_1'S_0'$$

$$Q_4 = S_2S_1S_0'$$

$$Q_5 = S_2S_1'S_0$$

$$Q_6 = S_2S_1S_0$$

$$Q_7 = S_2'S_1'S_0$$

Building a 3 - to - 8 decoder

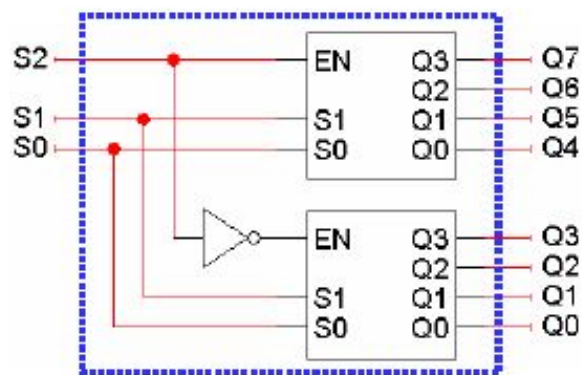
- Notice some patterns in the table below:
 - When $S_2 = 0$, outputs Q_0 - Q_3 are generated as in a 2-to-4 decoder.
 - When $S_2 = 1$, outputs Q_4 - Q_7 are generated as in a 2-to-4 decoder.

S2	S1	S0	Q0	Q1	Q2	Q3	Q4	Q5	Q6	Q7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Can you think of a 'hierarchical' design for this decoder, like we did with half and full adders?

Hierarchical decoder design

- Here’s a 3-to-8 decoder built from two smaller 2-to-4 decoders.
- When $S_2=0$, the bottom 2-to-4 decoder is enabled and generates a 1 for one of outputs Q_0 , Q_1 , Q_2 or Q_3 .
- When $S_2=1$, the top 2-to-4 decoder is enabled instead, and a 1 will be output for either Q_4 , Q_5 , Q_6 or Q_7 .



S2	S1	S0	Q0	Q1	Q2	Q3	Q4	Q5	Q6	Q7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Verify this circuit: use equations for the 2-to-4 decoder to derive equations for 3-to-8 decoder!

Notice anything special about the 3-to-8 decoder output expressions?



S2	S1	S0	Q0	Q1	Q2	Q3	Q4	Q5	Q6	Q7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

$$Q0 = S2'S1'S0'$$

$$Q1 = S2'S1'S0$$

$$Q2 = S2'S1 S0'$$

$$Q3 = S2'S1 S0$$

$$Q4 = S2 S1'S0'$$

$$Q5 = S2 S1'S0$$

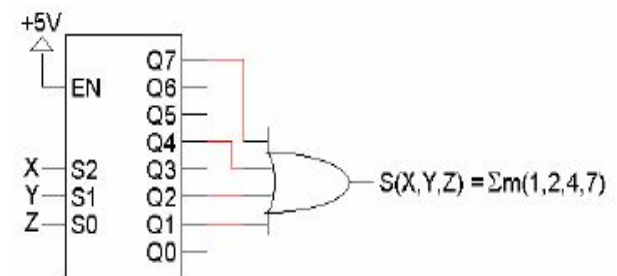
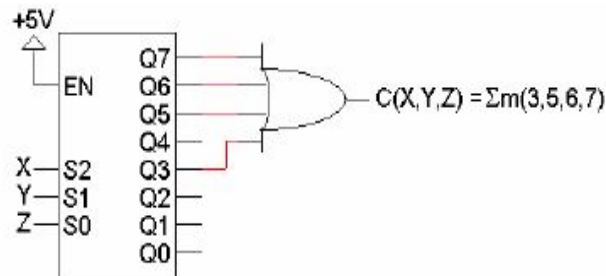
$$Q6 = S2 S1 S0'$$

$$Q7 = S2 S1 S0$$

Example: building an adder with decoder(s)



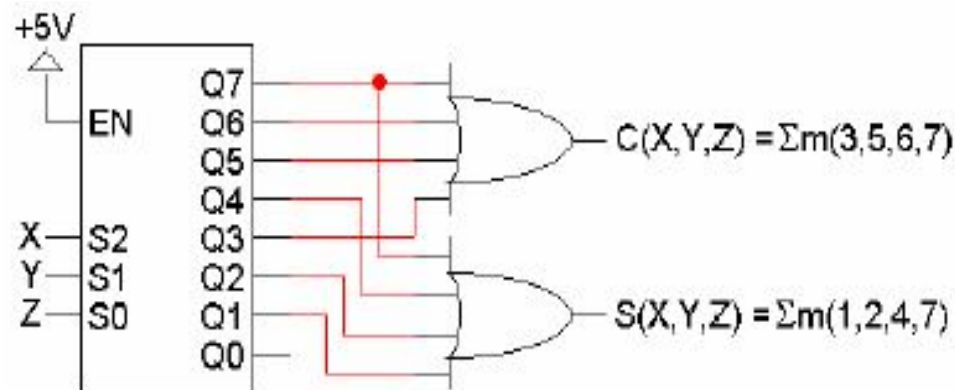
X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Verify:

$$\begin{aligned}
 S(X,Y,Z) &= Q_1 + Q_2 + Q_4 + Q_7 \\
 &= S_2'S_1'S_0 + S_2'S_1S_0' + S_2S_1'S_0' + S_2S_1S_0 \\
 &= X'Y'Z + X'YZ' + XY'Z' + XYZ \\
 &= m_1 + m_2 + m_4 + m_7
 \end{aligned}$$

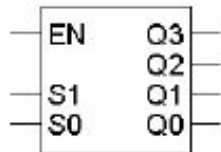
- Reuse the same decoder for both outputs:
- (Dis)advantages of this approach?



Active-high and active-low decoders

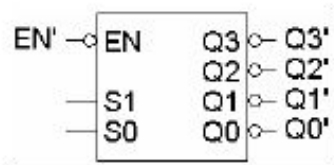


- The decoders we've seen so far are **active-high** decoders.



EN	S1	S0	Q0	Q1	Q2	Q3
0	x	x	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

- An **active-low decoder** is similar, but with an inverted EN input *and* inverted outputs—exactly one of the outputs will be *false*.

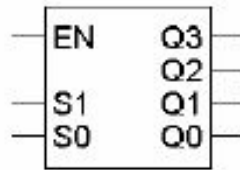


EN'	S1	S0	Q0'	Q1'	Q2'	Q3'
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0
1	x	x	1	1	1	1

Active-low decoders are maxterm generators!

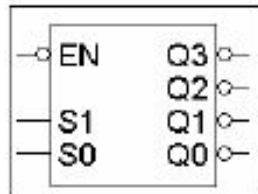


- Active-high decoders generate minterms, as we've already seen.



$$\begin{aligned}Q3 &= S1 S0 \\Q2 &= S1 S0' \\Q1 &= S1' S0 \\Q0 &= S1' S0'\end{aligned}$$

- The output equations for an active-low decoder are mysteriously similar, yet somehow different.



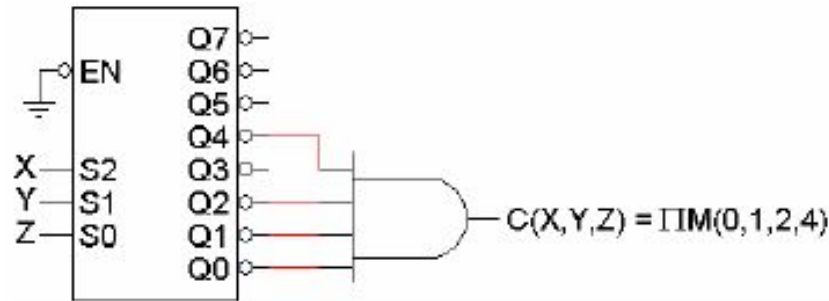
$$\begin{aligned}Q3' &= (S1 S0)' = S1' + S0' \\Q2' &= (S1 S0')' = S1' + S0 \\Q1' &= (S1' S0)' = S1 + S0' \\Q0' &= (S1' S0')' = S1 + S0\end{aligned}$$

- It turns out that active-low decoders generate **maxterms**, so they can implement functions as sums of maxterms.

Active-low decoder example



- Here is the carry function C implemented with an active-low decoder.
 - Recall from our algebra lecture that $\Sigma m(3,5,6,7) = \Pi M(0,1,2,4)$.
 - We need an AND gate to generate a *product* of sums.



- The “ground” symbol connected to the EN input represents logical 0, so this decoder is always enabled.

Encoders

- Opposite function as that of a decoder
- Takes 2^n inputs, and generates the binary equivalent of the input rank as output

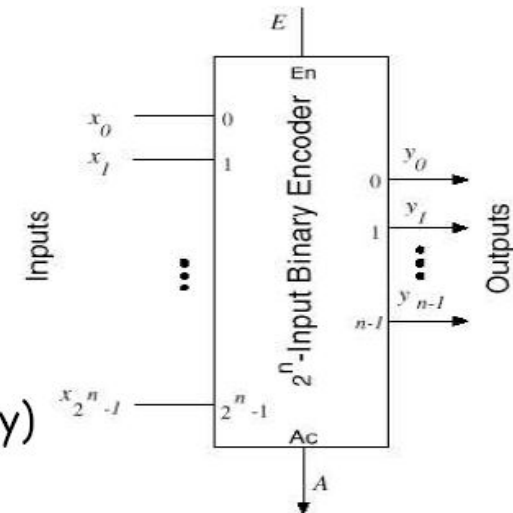
Example: 8-to-3 binary encoder (octal-to-binary)

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$A_0 = D_1 + D_3 + D_5 + D_7$$

$$A_1 = D_2 + D_3 + D_6 + D_7$$

$$A_2 = D_4 + D_5 + D_6 + D_7$$



Encoder design issues

Two ambiguities associated with the design of the simple encoder shown previously:

1. It assumes only one input to be active. If two inputs are simultaneously active, output becomes invalid.
2. $D_0 = 1$ or all $D_s = 0$ will both generate an output of 0

Example: 8-to-3 binary encoder (octal-to-binary)

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$A_0 = D_1 + D_3 + D_5 + D_7$$

$$A_1 = D_2 + D_3 + D_6 + D_7$$

$$A_2 = D_4 + D_5 + D_6 + D_7$$

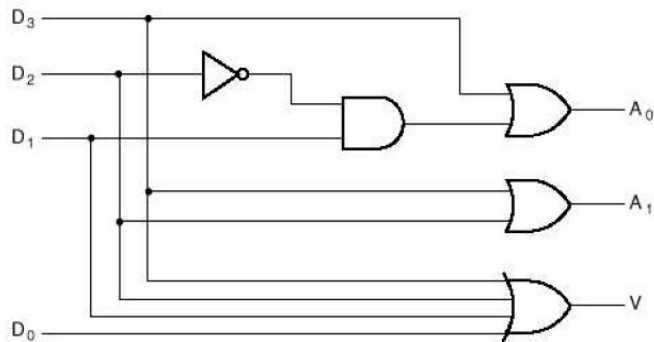
Solution: Priority encoders

- Multiple asserted inputs allowed; one has priority over others
- Separate indication of no-asserted-inputs

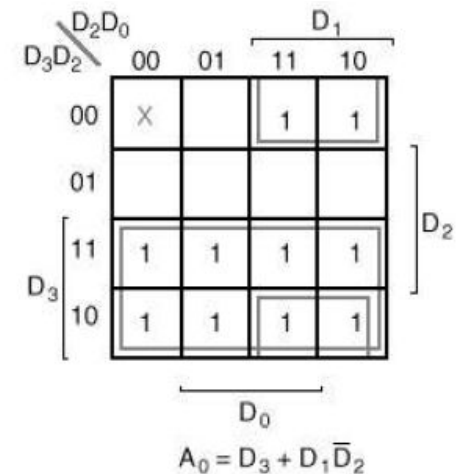
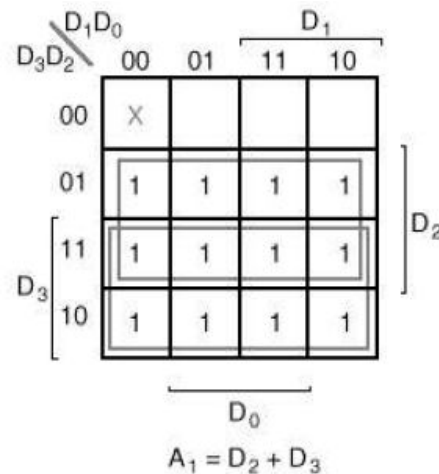
Example: 4-to-2 priority encoder



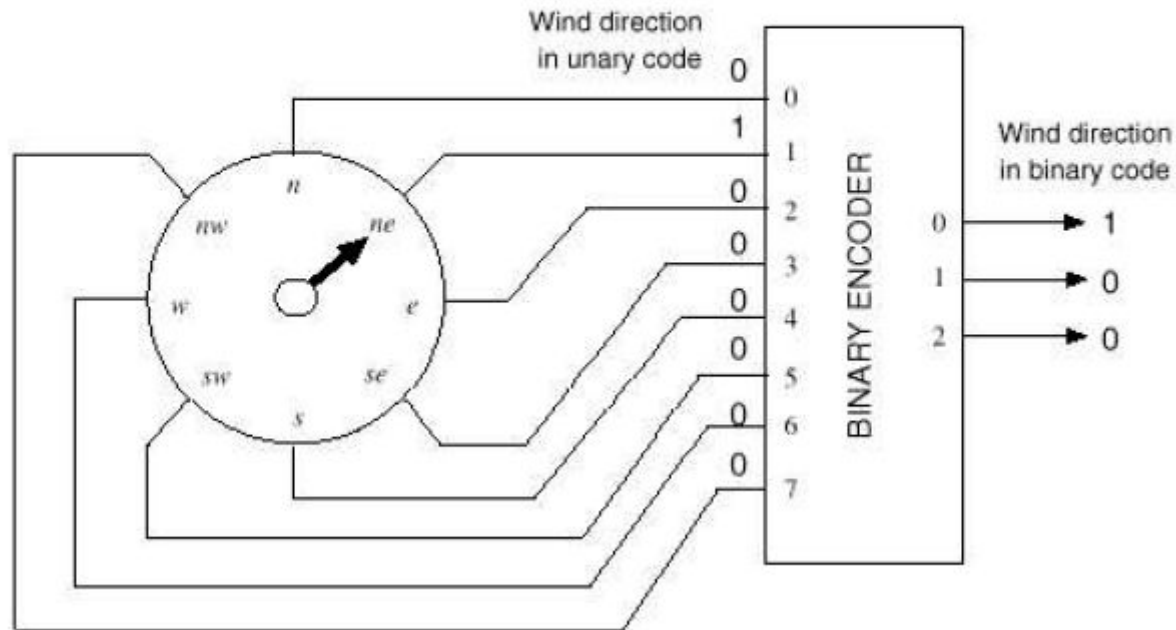
- If two or more inputs are simultaneously 1, output will correspond to the rank of the highest priority input
- 'Valid' output indicator, set to 1 only if at least one of the inputs is 1



Inputs				Outputs		
D_3	D_2	D_1	D_0	A_1	A_0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1



Use of binary encoder (example)



Uses of priority encoders (example)

